National University of Computer & Emerging Sciences Karachi Campus

AS-7: AegisSwarm-7

Autonomous Drone Swarm Defense System

Project Report

Object Oriented Programming
Section: BCS-2E

Group Members:
25K-0667 Tehreem Anwar

Abstract

AegisSwarm-7 (AS-7) is a program designed in C++ language to enhance air defense systems through a swarm of autonomous drones. Unlike traditional defense systems, AS-7 is a cost-friendly decentralized system that uses a many-to-many swarm logic to efficiently counter enemy threats. This project aims to improve existing air defense systems, to ensure the safety and security of the civilian population by prioritizing the conservation of critical infrastructure such as hospitals and power plants.

This project focuses mainly on being heavy in digital logic and programming rather than hardware. It uses physics and smart target calculations to counter threats. The project demonstrates a considerable and proficient understanding of a professional management system and reflects real world problems.

Built on the core pillars of Object Oriented Programming (OOP), the system uses the concepts of Inheritance, Aggregation, and Composition to efficiently simulate a battlefield grid where each drone operates as an independent entity, making AS-7 a decentralized system.

The WarRoom simulation initiates by establishing a battlefield grid, where the system first loads strategic infrastructure data to define the "No-Fire Zones." Then the necessary defense assets are loaded including the Kamikaz, Jammer and Sniper drones. Threats, which include Ballistic Missiles, Loitering Munitions and Electronic Decoys, are spawned in order to formally begin the battle simulation. Each drone is specialized for a specific threat: Kamikaz counters high-speed missiles, Jammers are for decoys, while Snipers take down drones. With the threat wave spawned, relevant interceptors are assigned to each incoming threat and the clock initiates. Throughout the engagement, the system executes a "Location Shield", where drones cross-check their impact coordinates with protected civilian sites to prevent collateral damage. Simultaneously, a real-time battery monitoring system manages the "Automatic Handover" logic, which replaces low-power units with fresh assets from the hangar.

AS-7 uses advanced C++ concepts, including virtual functions for polymorphic behavior, operator overloading for coordinate calculation and communication processing, and friend functions for collision logic. These features ultimately make this project more efficient and effective.

1. Project Overview

1.1 Introduction

Modern defense systems have undergone a fundamental shift. Militaries no longer rely only on expensive fighter jets or ballistic missiles. New strategies have emerged with battlefields using low-cost loitering munitions and suicide drones with the aim to overwhelm traditional defense systems both tactically and economically. Firing an interceptor missile worth $200, 000, 000 at a cheap, $20, 000 drone is not a victory, but rather an economic loss.

AS-7, short for AegisSwarm-7, is a simulation designed in C++ of an Autonomous-Swarm system designed to address this issue. Rather than relying on a single expensive interceptor, AS-7 uses a decentralized swarm of low-cost drones, each independently capable of tracking, targeting, and neutralizing incoming aerial threats. The system classifies every threat by type and risk level, dispatches appropriately classified interceptor units. Each drone continuously monitors its own health, replacing any low-powered drones mid-engagement through an Automatic Handover mechanism.

Built entirely on Object Oriented Programming principles, AS-7 uses concepts like Inheritance, Association and Polymorphism, demonstrating how these can be combined to address a real-world defense scenario using software.

Other than just interception, AS-7 incorporates a Location Shield that protects civilian infrastructure such as hospitals and power plants. The swarm also includes inter-drone communication, making the system decentralized. The result is a console-based simulation that executes threat interception using decision-making, resource management, and ethical constraints making it an efficient and effective autonomous defense network.

1.2 Problem Statement

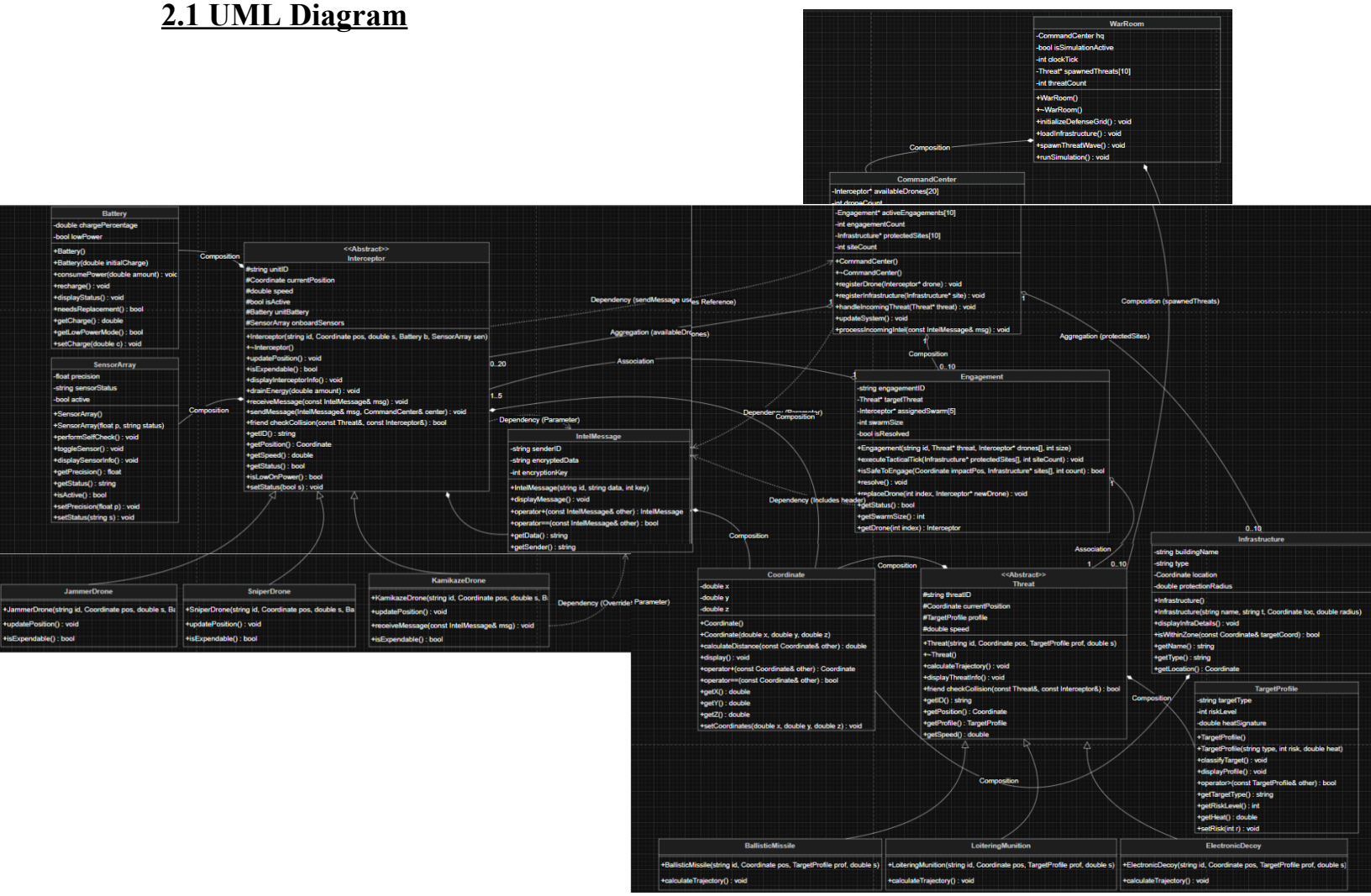
Current air defense systems, like HQ-9P (Pakistan) and Bavar-373 (Iran), are designed to target jets and missiles, making them powerful but extremely expensive. The problem which arises is that while the radar can detect into a wide range, these missile-focused systems can only engage around 6-8 targets simultaneously. Now if an enemy sends 30 low-cost drones, these systems will be overwhelmed and would not be able to shoot all 30 down. This would also be inefficient since these expensive missiles would be used to shoot down 30 cheap drones. Additionally, these defense systems are centralized with the communication happening through a ground base location, which reduces efficiency.

1.3 Objectives

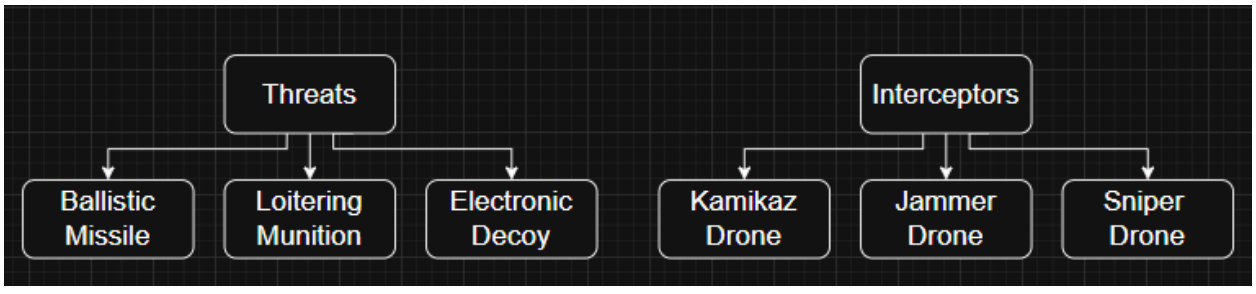
- To simulate a decentralized, autonomous drone swarm defense system capable of countering multiple aerial threats simultaneously.
- To implement a smart target prioritization system that assigns specialized interceptors based on threat risk levels and heat signatures.
- To ensure protection of critical civilian infrastructure using a "Location Shield" through a trajectory-nudge logic.
- To manage swarm drones efficiently through an automatic handover protocol that swaps low-powered units with fresh assets.
- To calculate precise intercept coordinates using the Euclidean distance formula and simulate path through trajectory physics (parabolic arcs / zig-zag patterns).
- To display a real-time, tick-by-tick tactical log outputting system updates, alerts and results.
- To reinforce Object Oriented Programming (OOP) concepts, including Inheritance, Polymorphism, Composition, and Aggregation.

2. System Design

2.1 UML Diagram



2.2 Hierarchy Flowchart



2.3 Component Breakdown

WarRoom:

- Serves as a simulation engine of AS-7.
- Owns the CommandCenter through Composition.
- Initializes battle grid and infrastructure, loads assets, and spawns threat waves.
- Drives the clock, advancing the battle tick by tick until either all engagements are resolved or the time limit is reached.

CommandCenter:

- Serves as the brain of AS-7.
- Maintains 3 core arrays: available interceptor drones, active engagements, and registered infrastructure.
- Handles incoming threat by applying Target Prioritization:
    - High-Risk: (>=9) Assigns 5 Kamikaz Drones
    - Medium-Risk:(>=5) Assigns 1 Sniper Drone
    - Low-Risk:(<5) Assigns 1 Jammer Drone
- Selected drones are proceeded to an Engagement.
- Keeps a check at each drone’s battery level and applies Automatic Handover accordingly.
- Efficiently keeps Engagement running even if a drone is low-powered.

Engagement:

- Serves as the main logic of AS-7.
- Binds one threat to its assigned swarm of interceptors.
- Calculates the threat's trajectory and each drone's position simultaneously.
- Checks coordinates to decide if the position of the drone is ready for collision.
- Before firing, the system requires a Location Shield check to prevent casualties.
- Resolves the threat according to each individual drone’s behaviour.

3. Technical Implementation

3.1 Inheritance & Abstract Classes

1. Attack

- **Abstract Class:** Threat
- **Derived Classes:**
  - Ballistic Missile: High Speed, High Risk, Parabolic Trajectory
  - Loitering Munition: Medium Speed, Medium Risk, Zig-Zag Trajectory
  - Electronic Decoy: Low Speed, Low Risk, “Distraction Noise”

2. Defense

- **Abstract Class:** Interceptor
- **Derived Classes:**
  - Kamikaz Drone: Mobile kinetic, Explodes, Physical Kill
  - Jammer Drone: Mobile electronic, Emits Signals, “Soft Kill”
  - Sniper Drone: Stationary, Fires, Distant Physical Kill

Relation:

- Ballistic Missiles are countered by mobile swarm of Kamikaz.
- Loitering Munitions are countered by singular base Sniper.
- Electronic Decoys are countered by singular mobile Jammer.

3.2 Strategic Logic

1. Threat Prioritization & Efficiency

- 1 Ballistic - 5 Kamikaz
- 1 Munition - 1 Sniper
- 1 Decoy - 1 Jammer

```
// TARGET PRIORITIZATION
int dronesNeeded = 0;
string requiredType = "";

if (risk >= 9) {
    dronesNeeded = 5;
    requiredType = "KAM";   // 5 kamikazes for a high-speed ballistic missile
} else if (risk >= 5) {
    dronesNeeded = 1;
    requiredType = "SNP";   // 1 sniper for a mid-tier loitering munition
} else {
    dronesNeeded = 1;
    requiredType = "JAM";   // 1 jammer for a low-risk electronic decoy
}
```

2. Battery Handover

- Battery Drain logic implements realistic power consumption.
- Maintains a check on each drone's battery status.
- Alerts if any one of the drones has battery < 15%.
- Deploys another drone to replace the current one and the Engagement continues.

```
// Logic for Handover: Replacement function
void Engagement::replaceDrone(int index, Interceptor* newDrone) {
    if (index >= 0 && index < swarmSize) {
        assignedSwarm[index] = newDrone;
    }
}
```

```
// 2. THE HANDOVER LOGIC: Check for battery alerts
for (int d=0; d<activeEngagements[i]->getSwarmSize(); d++) {
    Interceptor* currentDrone = activeEngagements[i]->getDrone(d);

    if (currentDrone != nullptr && currentDrone->isLowOnPower()) {
        cout<<"[ALERT]: Drone "<<currentDrone->getID()<<" battery low! Initiating Handover."<<endl;

        // Step A: Find out what type we need (First 3 chars: "KAM", "SNP", "JAM")
        string typeNeeded = currentDrone->getID().substr(0, 3);
        Interceptor* freshReplacement = nullptr;
        int replacementIndex = -1;

        // Step B: Search Hangar for a fresh drone of the same type
        for (int k=0; k<droneCount; k++) {
            if (availableDrones[k]->getID().substr(0, 3) == typeNeeded && !availableDrones[k]->isLowOnPower()) {
                freshReplacement = availableDrones[k];
                replacementIndex = k;
                break;
            }
        }
```

```
            // Step C: If fresh drone found, swap them
            if (freshReplacement != nullptr) {
                cout<<"[HANDOVER]: Swapping "<<currentDrone->getID()<<" with "<<freshReplacement->getID()<<endl;

                activeEngagements[i]->replaceDrone(d, freshReplacement);

                // Put the old drone back in the hangar (to be "recharged")
                currentDrone->setStatus(false); // Set to Inactive
                availableDrones[replacementIndex] = currentDrone;

            } else {
                cout<<"[CRITICAL]: No fresh "<<typeNeeded<<" units available in hangar!"<<endl;
            }
        }
    }
```

3. Location Shield

- Before execution of firing protocol, Location Shield is checked.
- Each infrastructure has a designated radius of protection.
- Ensure impact is not in proximity of any critical infrastructure.
- Usage of distance calculation through coordinate system.
- If impact lies in the region, the system is alerted. IntelMessage notifies the drone.
- The specific drone redirects its path to a different coordinate.
- This logic only applies to Kamikaz Drone since it is the only asset which counters threats by physical collision.

```
// Function to carry out Location Shield logic utilizing Infrastructure class
bool Engagement::isSafeToEngage(Coordinate impactPos, Infrastructure* sites[], int count) const {
    for (int i = 0; i < count; i++) {
        // Calling the Infrastructure class to check its own protection radius
        if (sites[i]->isWithinZone(impactPos)) {
            cout<<"[SHIELD]: DANGER! Impact within No-Fire Zone of "<<sites[i]->getName()<<endl;
            return false;   // Within No-Fire Zone: not clear for impact
        }
    }
    return true;    // No infrastructure nearby: clear for impact
}
```

```
void KamikazeDrone::receiveMessage(const IntelMessage& msg) {
    if (msg.getData() == "REDIRECT_IMPACT") {
        // Nudge Logic: Adjust speed and shift coordinates to avoid infrastructure
        speed = speed * 0.8;
        currentPosition.setCoordinates(currentPosition.getX() + 20, currentPosition.getY() + 20, currentPosition.getZ());
        cout<<"[ALERT]: Drone "<<unitID<<" performing kinetic nudge. New path calculated."<<endl;
    } else {
        Interceptor::receiveMessage(msg);
    }
}
```

3.3 Physics & Trajectories

1. Ballistic Missile: The Parabolic Arc

Missiles move at high speeds and are influenced by gravity. I have used a simplified version of kinematic equations:

- X-axis (Horizontal):

$$x = x' - (v \cdot \Delta t)$$

  This represents constant horizontal velocity toward the target.

- Y-axis (Vertical):

$$y = y' - (0.5.\ g\ .\ \Delta t^2)$$

  This simulates a "gravity drop." Even though a missile is powered, it follows a curved path (an arc) as it approaches the ground.

- $t = 0.1$ for Ballistic Missiles in order to correctly observe the parabolic path.

2. Loitering Munition: The Zig-Zag Logic

Drones don't move in straight lines because they want to avoid being intercepted.

- **The Variable (static bool zig):** We use a static boolean because it remembers its value between function calls. If it was true last time, it becomes false this time.
- **The Shift:**
  - If zig is true:

$$y = y' + 10$$

  - If zig is false:

$$y = y' - 10$$

- **The Result:** On every update, the drone moves forward on the X-axis but "bounces" up and down on the Y-axis. This creates a zig-zag pattern that makes it harder for a drone to predict the exact intercept point.
- $t = 0.05$ for Loitering Munitions in order to correctly observe the zig-zag path.

3. Electronic Decoy: Linear Movement

Decoys follow a simple path. They are usually dropped from an aircraft or launched to fly in a straight path to distract sensors.

- **Formula:**

$$x = x' - (v\ .\ \Delta t)$$

- Its Y and Z coordinates remain perfectly constant. This allows Target Prioritization to identify this as a low threat and not to waste resources on this.
- $t = 0.02$ for Electronic Decoys in order to correctly confirm the linear path.

4. Simulation & Tactical Results

Scenario 1: Successful Neutralization

- **Threat:** Ballistic Missile BM-67
- **Interceptor:** 5 units of Kamikaz Drones KAM-15 to KAM-11
- **Result:** Successful Neutralization
- **Time:** 2 Ticks
- **Initialization:**

```
Threat* missile  = new BallisticMissile("BM-67", Coordinate(40,0,0), highRisk, 10.0);
```

```
for(int i = 0; i < 15; i++) {
    hq.registerDrone(new KamikazeDrone("KAM-" + to_string(i+1), Coordinate(0,0,0), 100.0, standardBattery, standardSensors));
}
```

- **Output:**

```
[COMMAND]: Deploying 5 KAM units for Threat BM-67
[ENGAGEMENT]: Initialized ENG-BM-67. Swarm of 5 drones assigned to Threat BM-67
```

```
--- CLOCK TICK: 1 ---
[SYSTEM]: Ballistic Missile BM-67 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-15 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-14 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-13 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-12 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-11 at X: 10 accelerating...
```

```
--- CLOCK TICK: 2 ---
[SYSTEM]: Ballistic Missile BM-67 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-15 at X: 20 accelerating...
[LOCATION SHIELD]: Area Clear. Finalizing Neutralization.
[DESTRUCTION]: Drone KAM-15 neutralized along with threat.
[DESTRUCTION]: Drone KAM-14 neutralized along with threat.
[DESTRUCTION]: Drone KAM-13 neutralized along with threat.
[DESTRUCTION]: Drone KAM-12 neutralized along with threat.
[DESTRUCTION]: Drone KAM-11 neutralized along with threat.
[ENGAGEMENT]: ENG-BM-67 RESOLVED.
[COMMAND]: Engagement 0 resolved. Clearing memory.
```

Scenario 2: Location Shield Logic

- **Threat:** Ballistic Missile HNT-01
- **Interceptor:** 5 units of Kamikaz Drones KAM-10 to KAM-06
- **Result:** Successful Neutralization through Kinetic Nudge
- **Time:** 6 Ticks
- **Initialization:**

```
Threat* hunt = new BallisticMissile("HNT-01", Coordinate(50, 0, 0), highRisk, 20.0);
```

```
for(int i = 0; i < 15; i++) {
    hq.registerDrone(new KamikazeDrone("KAM-" + to_string(i+1), Coordinate(0,0,0), 100.0, standardBattery, standardSensors));
}
```

- **Output:**

```
[COMMAND]: Deploying 5 KAM units for Threat HNT-01
[ENGAGEMENT]: Initialized ENG-HNT-01. Swarm of 5 drones assigned to Threat HNT-01
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-8 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-7 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-6 at X: 20 accelerating...
```

```
--- CLOCK TICK: 3 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-10 attempting trajectory nudge.
[ALERT]: Drone KAM-10 performing kinetic nudge. New path calculated.
```

```
--- CLOCK TICK: 4 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 58 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-9 attempting trajectory nudge.
[ALERT]: Drone KAM-9 performing kinetic nudge. New path calculated.
[ALERT]: Drone KAM-10 battery low! Initiating Handover.
[HANDOVER]: Swapping KAM-10 with KAM-1

--- CLOCK TICK: 5 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-1 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 58 accelerating...
[STATUS]: Kamikaze Drone KAM-8 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-8 attempting trajectory nudge.
[ALERT]: Drone KAM-8 performing kinetic nudge. New path calculated.
[ALERT]: Drone KAM-9 battery low! Initiating Handover.
[HANDOVER]: Swapping KAM-9 with KAM-2

--- CLOCK TICK: 6 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-1 at X: 20 accelerating...
[LOCATION SHIELD]: Area Clear. Finalizing Neutralization.
[DESTRUCTION]: Drone KAM-1 neutralized along with threat.
[DESTRUCTION]: Drone KAM-2 neutralized along with threat.
[DESTRUCTION]: Drone KAM-8 neutralized along with threat.
[DESTRUCTION]: Drone KAM-7 neutralized along with threat.
[DESTRUCTION]: Drone KAM-6 neutralized along with threat.
[ENGAGEMENT]: ENG-HNT-01 RESOLVED.
[COMMAND]: Engagement 0 resolved. Clearing memory.
```

Scenario 3: Automatic Handover

- **Threat:** Ballistic Missile HNT-01
- **Interceptor:** 5 units of Kamikaz Drones KAM-10 to KAM-06
- **Result:** Successful Neutralization through Automatic Handover with KAM-01 & KAM-02 for KAM-10 & KAM-09.
- **Time:** 6 Ticks

```
Threat* hunt = new BallisticMissile("HNT-01", Coordinate(50, 0, 0), highRisk, 20.0);
```

```
for(int i = 0; i < 15; i++) {
    hq.registerDrone(new KamikazeDrone("KAM-" + to_string(i+1), Coordinate(0,0,0), 100.0, standardBattery, standardSensors));
}
```

- **Initialization:**
- **Output:**

```
[COMMAND]: Deploying 5 KAM units for Threat HNT-01
[ENGAGEMENT]: Initialized ENG-HNT-01. Swarm of 5 drones assigned to Threat HNT-01
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-8 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-7 at X: 20 accelerating...
[STATUS]: Kamikaze Drone KAM-6 at X: 20 accelerating...
```

```
--- CLOCK TICK: 3 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-10 attempting trajectory nudge.
[ALERT]: Drone KAM-10 performing kinetic nudge. New path calculated.
```

```
--- CLOCK TICK: 4 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-10 at X: 58 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-9 attempting trajectory nudge.
[ALERT]: Drone KAM-9 performing kinetic nudge. New path calculated.
[ALERT]: Drone KAM-10 battery low! Initiating Handover.
[HANDOVER]: Swapping KAM-10 with KAM-1

--- CLOCK TICK: 5 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-1 at X: 10 accelerating...
[STATUS]: Kamikaze Drone KAM-9 at X: 58 accelerating...
[STATUS]: Kamikaze Drone KAM-8 at X: 30 accelerating...
[SHIELD]: DANGER! Impact within No-Fire Zone of Jinnah Hospital
[ALERT]: Impact point too close to Infrastructure! Drone KAM-8 attempting trajectory nudge.
[ALERT]: Drone KAM-8 performing kinetic nudge. New path calculated.
[ALERT]: Drone KAM-9 battery low! Initiating Handover.
[HANDOVER]: Swapping KAM-9 with KAM-2

--- CLOCK TICK: 6 ---
[SYSTEM]: Ballistic Missile HNT-01 updating arc trajectory.
[STATUS]: Kamikaze Drone KAM-1 at X: 20 accelerating...
[LOCATION SHIELD]: Area Clear. Finalizing Neutralization.
[DESTRUCTION]: Drone KAM-1 neutralized along with threat.
[DESTRUCTION]: Drone KAM-2 neutralized along with threat.
[DESTRUCTION]: Drone KAM-8 neutralized along with threat.
[DESTRUCTION]: Drone KAM-7 neutralized along with threat.
[DESTRUCTION]: Drone KAM-6 neutralized along with threat.
[ENGAGEMENT]: ENG-HNT-01 RESOLVED.
[COMMAND]: Engagement 0 resolved. Clearing memory.
```

5. Conclusion

AegisSwarm-7 successfully demonstrates an autonomous drone swarm system, which provides a more efficient, secure and cost-effective alternative to traditional, centralized air defense. AS-7 uses Decentralized Swarm Logic to ensure maximum efficiency, ensuring that even if one drone goes down, another drone is there to take its place and keep the defense moving. By using Target Prioritization, the system ensures resources are not being wasted and are assigned to their respective threats. This proves to be highly effective and cost-friendly. The Location Shield Logic ensures security of critical infrastructure and protection of civilians. In today's era of political unrest and global instability, countries like Pakistan and Iran need to upgrade their air defense systems so that it proves to be secure and economical.

6. Future Enhancements

To take AS-7 to the next level and improve its interface, we can implement the following improvements:

- **3-D Graphical Interface (GUI):** Enhancing the console-based output to a more realistic 3-D visualization using OpenGL to improve monitoring of drone-threat engagements.
- **Advanced Environmental Physics:** Incorporating external variables such as air resistance, atmospheric drag, and varying weather conditions to test the system with more accurate trajectory calculations.
- **Database Integration:** Implementing an **SQLite database** to log comprehensive battle histories, battery performance metrics, and engagement stats for post-mission analysis.

7. Tools and Technologies

Programming Language: C++ (OOP)
Framework: Standard Template Library (STL)
Development Environment: Dev C++, VS Code, Programiz, Github
Operating System: Windows 11

8. References:

- wikipedia.com
- canva.com
- draw.io
- How to Program C++ by Paul Deitel & Harvey Deitel